Priscilla Dankwa
Corpus Linguistics

Github Repository Link : https://github.com/pdankwa-UKN/Corpus-Linguistics-Project--Akan

AKAN CORPUS ANALYSIS TOOLKIT

Overview

This documentation takes us through a python package designed to compile a chapter from the Akan online Bible. The toolkit to scrapes it into a clean dataset and performs corpus organisation, searching, annotation, frequency analysis, lexical diversity analysis, concordancing, and collocation exploration.

Corpus Description

The corpus is from Psalm 119 in Akan (Twi), as published in the New World Translation on jw.org. It's written in the extended Latin alphabet Akan for akan which includes letters the characters ɛ and ɔ.
The verses are pulled from https://www.jw.org/tw/nhomakorabea/bible/nwt/nhoma/nnwom/119/, the Akan New World Translation page for Psalm 119. The urllib request fetches the page HTML, and BeautifulSoup picks out every element along with the page title, saving the raw text into scraped verses.json. Then there's a second scrape of jw.org's homepage using trafilatura and BeautifulSoup string-matching to capture "license" information ie; title, publisher, and date, which gets written to metadata.json.

System Architecture & Design Decisions

The package is built as a linear pipeline, with each stage reading a file and writing a new one, this makes it easy to re-run any single step without redoing the whole thing. The code uses both regex and re, since regex supports \p{L} for matching Unicode letters properly.
The search and analysis steps are also interactive, prompting the user for input via input(), making corpus analysis for each step easier

The stages are:

1. Scrape verses: scrpaes webpage →scraped verses.json
2. Extract metadata : →metadata.json
3. Segment and tokenize (segment_and_tokenize_verses): cleans up stray scraping artifacts (\, *, +, stray quotes), splits the raw text into sentences on periods, strips leading verse numbers, and re-tokenizes each sentence →tokens_segmented.txt
4. Remove stopwords (create_clean_verses) : filters out common Akan function words (stored in akan_stopwords.json) and cleans corpus →final_verses.txt

5. Search the corpus (corpus_search) : search function handles plain keywords, regex patterns, and special annotation, returns matches with 4 words of context on either side → search_results.txt
6. Run comprehensive analysis (comprehensive_analysis) — computes unigram, bigram, and trigram frequencies, character frequencies, and collocation statistics (PMI, t-score, Dice coefficient), with an interactive menu to either run the full report or filter everything down to one target word → comprehensive_analysis.txt
7. Concordance & diversity check (corpus_analysis) — calculates the Type-Token Ratio (a measure of vocabulary diversity) and builds a KWIC-style concordance table for a chosen word → search_analysis_results.txt

Usage Instructions

You'll need beautifulsoup4, requests, trafilatura, lxml_html_clean and regex installed.
Run the cells in order, top to bottom:

1. Run the two scraping cells to produce scraped verses.json and metadata.json.
2. Run the segmentation cell to produce tokens_segmented.txt.
3. Run the stopword-removal cell to produce final_verses.txt.
4. Run the corpus search cell - interactive mode
5. Run the comprehensive analysis cell - interactive mode (full report, focused word analysis, or exit)
6. Run the corpus analysis cell - interactive mode (TTR and KWIC)

Example Commands and Output

Segmentation and tokenization : segment_and_tokenize_verses("scraped verses.json", "tokens_segmented.txt")

Stopword Removal : create_clean_verses("tokens_segmented.txt", "final_verses.txt")

Corpus Search: corpus_search("final_verses.txt", "search_results.txt", user_query)

Challenges Faced

The first challenge was finding a site to scrape, a number of sites with Akan text either blocked scraping outright (I couldn't figure out how to do it with the developer tools despite several other people trying to help) or they didn't have any informational metadata kind of like jw.org but that was the best option as it was the only one I could figure out.
After that challenge, there was also a difficulty with getting metadata and after numerous web searches I ended importing a second Python library, trafilatura, specifically to extract the metadata even then some details couldn't be found and so were manually provided.

A third challenge was that resources for Akan was very scarce. There were very few existing tools built for the language. eg. part-of-speech and morpheme tagger.
One tool (Hugging face) looked promising but was far too large to realistically download and work with on my machine, so it had to be abandoned and taggers worked on manually